

CI/CD Integration & GitOps

Automating Kubernetes Delivery Pipelines — Module
13 of 13

Enterprise EKS Platform





Agenda

Concepts

- CI/CD fundamentals for Kubernetes
- GitOps principles and models
- Push vs pull-based deployments

Tools & Patterns

- ArgoCD deep dive
- FluxCD overview
- Pipeline tooling (GitHub Actions, Tekton)
- Deployment patterns and promotion strategies

Lab 13: GitOps with ArgoCD — declarative application delivery, sync, and rollback (~30 minutes)



CI/CD Fundamentals for Kubernetes

From code commit to running containers

Traditional CI/CD vs Cloud-Native CI/CD

Traditional CI/CD

- Deploy artifacts (JARs, binaries) to VMs
- Configuration management tools (Ansible, Chef)
- Mutable infrastructure — update in place
- Snowflake servers over time
- Rollback = redeploy previous version

Cloud-Native CI/CD

- Deploy container images to orchestrators
- Declarative manifests (YAML, Helm, Kustomize)
- Immutable infrastructure — replace, never patch
- Identical environments from images
- Rollback = point to previous image tag

Key shift: The artifact is a container image, and the desired state is a

Pipeline Stages for Kubernetes

End-to-End Pipeline Flow

Code Commit → Build → Test → Scan → Push → Deploy → Verify

- **Build** — compile code, build container image
- **Test** — unit tests, integration tests, contract tests
- **Scan** — vulnerability scanning, policy compliance
- **Push** — tag and push image to registry (ECR, GHCR)
- **Deploy** — update manifests, sync to cluster
- **Verify** — smoke tests, health checks, monitoring

Separation of concerns: CI builds and tests the artifact. CD deploys the artifact. GitOps manages the desired state.

Container Image Lifecycle

Build

- Multi-stage Dockerfiles
- Minimal base images (distroless, Alpine)
- Build cache optimization

Tag & Push

- Immutable tags (Git SHA, semver)
- Never use :latest in deployments
- Push to ECR / private registry

Scan & Sign

- Trivy / Snyk vulnerability scan
- Cosign for image signing
- Admission control for verified images

```
# Example: Build, scan, tag, push
docker build -t myapp:${GIT_SHA} .
trivy image --severity HIGH,CRITICAL myapp:${GIT_SHA}
cosign sign --key cosign.key myapp:${GIT_SHA}
ECR=123456789.dkr.ecr.us-east-2.amazonaws.com
docker tag myapp:${GIT_SHA} ${ECR}/myapp:${GIT_SHA}
docker push ${ECR}/myapp:${GIT_SHA}
```



GitOps Principles

Git as the single source of truth for infrastructure

GitOps: Definition & Four Principles

What is GitOps?

An operational framework that applies DevOps practices for infrastructure automation — using Git as the single source of truth for declarative infrastructure and applications.

1. Declarative

The entire system is described declaratively (YAML manifests, Helm charts, Kustomize overlays)

2. Versioned & Immutable

The canonical desired state is versioned in Git, providing a complete audit trail

3. Pulled Automatically

Agents automatically pull the desired

4. Continuously Reconciled

Agents observe actual state, detect

Push vs Pull-Based Deployment Models



Push-Based (Traditional)

- CI/CD tool runs `kubectl apply`
- Requires cluster credentials in CI
- No drift detection after deploy
- Pipeline is the only deployment path
- Examples: Jenkins, GitHub Actions (direct)



Pull-Based (GitOps)

- Agent in cluster polls Git for changes
- No cluster credentials outside cluster
- Continuous drift detection and repair
- Git commit is the deployment trigger
- Examples: ArgoCD, FluxCD

Security advantage: Pull-based models eliminate the need to store cluster credentials in external CI systems — the agent only needs read

Jenkins CI/CD Pipeline Integration



Jenkinsfile Pipeline Stages

- **Build** — compile code, build container image, push to ECR
- **Test** — unit tests, integration tests, code coverage
- **Scan** — image vulnerability scanning (Trivy, ECR scan)
- **Deploy** — `kubectl apply` or `helm upgrade` to EKS



Kubernetes Plugin

- Dynamic build agents launched as Pods in EKS
- Each build gets an isolated, ephemeral environment
- Pod templates define containers (maven, docker, kubectl)
- Scales to zero when idle — no persistent agent VMs

EKS integration: Jenkins authenticates to EKS via IRSA (IAM Roles for Service Accounts) or kubeconfig with a service account token — avoid storing long-lived cluster admin credentials

From Jenkins Push to GitOps Pull

Hybrid Pattern: Jenkins CI + GitOps CD

```
Developer → Git Push → Jenkins Pipeline
├── Build → Container Image → ECR
├── Test → Unit + Integration
├── Scan → Trivy / ECR Scanning
├── Update → Image tag in GitOps repo
└──
    ↓
    ArgoCD detects change → Deploys to EKS
```

Best of both worlds: Jenkins handles CI (build, test, scan) while ArgoCD owns CD (deploy, reconcile, rollback) — no cluster credentials leave the cluster.

Git as Single Source of Truth

Repository Structure Patterns

```
gitops-repo/  
├── apps/  
│   ├── frontend/  
│   │   ├── base/           # deployment.yaml, service.yaml, kustomization.yaml  
│   │   └── overlays/       # dev/, staging/, production/  
│   └── backend/  
│       ├── base/  
│       └── overlays/  
├── infrastructure/         # cert-manager/, ingress-nginx/, monitoring/  
└── clusters/               # dev-cluster/, staging-cluster/, prod-cluster/
```

Every change goes through a pull request — code review, approval, and audit trail are built into the workflow.



Declarative, GitOps continuous delivery for Kubernetes

ArgoCD Architecture

API Server

- Web UI dashboard
- gRPC/REST API
- RBAC enforcement
- SSO integration (OIDC, SAML)

Repo Server

- Clones Git repositories
- Renders manifests (Helm, Kustomize)
- Caches rendered output
- Stateless & scalable

Application Controller

- Watches live cluster state
- Compares desired vs actual
- Executes sync operations
- Reports health status

ArgoCD Application CRD

```
kind: Application
metadata: { name: frontend, namespace: argocd }
spec:
  source:
    repoURL: https://github.com/mycompany/gitops-repo.git
    targetRevision: main
    path: apps/frontend/overlays/staging
  destination:
    server: https://kubernetes.default.svc
    namespace: frontend
  syncPolicy:
    automated: { selfHeal: true, prune: true }
    # ... retry, syncOptions, etc.
```

Source + Destination + SyncPolicy = the three pillars of every ArgoCD Application definition.

Sync Policies



Automated Sync

- Auto-deploy on Git commit
- No manual approval needed
- Ideal for dev/staging
- Optional for production



Self-Heal

- Revert manual `kubectl` changes
- Detect and fix drift
- Ensures Git remains truth
- Prevents configuration drift



Prune

- Delete resources removed from Git
- Prevents orphaned resources
- Use with caution in prod
- Combine with Foreground deletion

Multi-Environment Management

Kustomize Overlays

```
# overlays/staging/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: [../../base]
patchesStrategicMerge:
  - replica-patch.yaml
  - resource-patch.yaml
images:
  - name: myapp
    newTag: v2.1.0
```

ApplicationSet

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: { name: frontend }
spec:
  generators:
    - list:
        elements:
          - { cluster: dev, url: https://dev.k8s.local }
          - { cluster: staging, url: https://staging.k8s.local }
          - { cluster: prod, url: https://prod.k8s.local }
  template:
    spec:
      source:
        path: apps/frontend/overlays/{{cluster}}
```

FluxCD Overview

A toolkit approach to GitOps on Kubernetes

FluxCD Architecture

Toolkit of Specialized Controllers

Source Controller

- Fetches from Git, Helm repos, S3
- Produces versioned artifacts
- Handles authentication

Kustomize Controller

- Applies Kustomize overlays
- Handles plain YAML too
- Dependency ordering

Helm Controller

- Manages HelmRelease CRDs
- Automated upgrades/rollbacks
- Values from ConfigMaps/Secrets

Additional controllers: Notification Controller (alerts & webhooks),

FluxCD GitRepository & Kustomization

GitRepository

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata: { name: app-repo, namespace: flux-system }
spec:
  interval: 1m
  url: https://github.com/mycompany/gitops-repo
  ref: { branch: main }
  secretRef: { name: git-credentials }
```

Kustomization

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata: { name: frontend, namespace: flux-system }
spec:
  interval: 5m
  sourceRef: { kind: GitRepository, name: app-repo }
  path: ./apps/frontend/overlays/prod
  prune: true
  healthChecks:
    - kind: Deployment
      name: frontend
      namespace: frontend
```

FluxCD vs ArgoCD Comparison

Feature	ArgoCD	FluxCD
Architecture	Monolithic application	Toolkit of controllers
UI	Rich web UI built-in	No built-in UI (Weave GitOps optional)
Multi-tenancy	Projects & RBAC	Namespace-scoped CRDs
Multi-cluster	ApplicationSets	Kustomization per cluster
Image automation	Argo Image Updater (separate)	Built-in image automation
CNCF Status	Graduated	Graduated

Both are excellent choices. ArgoCD excels with its UI and multi-cluster management. FluxCD excels with its lightweight, composable design and native image automation.



FluxCD in Practice

Bootstrap, HelmReleases, Image Automation & Multi-Tenancy



Flux Bootstrap & Repo Structure

```
# Bootstrap Flux on a new cluster
flux bootstrap github \
  --owner=mycompany \
  --repository=fleet-infra \
  --branch=main \
  --path=clusters/production \
  --personal=false
```

Recommended Repository Structure

```
fleet-infra/
├── clusters/
│   ├── production/
│   │   ├── flux-system/      # Auto-generated
│   │   ├── infrastructure.yaml
│   │   └── applications.yaml
│   └── staging/
├── infrastructure/          # Shared infra components
└── applications/            # Application manifests
```



HelmRelease CRD

```
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata: { name: nginx-ingress, namespace: infrastructure }
spec:
  interval: 15m
  chart:
    spec:
      chart: ingress-nginx
      version: "4.x" # Semver range
      sourceRef:
        kind: HelmRepository
        name: ingress-nginx
  values:
    controller:
      replicaCount: 3
      service: { type: LoadBalancer }
    upgrade: { remediation: { retries: 3 } }
    rollback: { cleanupOnFail: true }
```

- **version** — semver ranges for auto minor/patch upgrades
- **upgrade.remediation** — auto-retry or rollback on failure
- **valuesFrom** — inject secrets from ConfigMaps/Secrets without storing in Git



Image Automation Pipeline

```
# 1. Scan registry for new tags
kind: ImageRepository
spec:
  image: ghcr.io/mycompany/my-app
  interval: 5m
---
# 2. Select latest matching tag
kind: ImagePolicy
spec:
  imageRepositoryRef: { name: app-image }
  policy: { semver: { range: ">=1.0.0 <2.0.0" } }
---
# 3. Write selected tag back to Git
kind: ImageUpdateAutomation
spec:
  sourceRef: { kind: GitRepository, name: app-manifests }
  update: { path: ./clusters/production, strategy: Setters }
```

Closed loop: CI builds image → Flux detects new tag → updates manifest in Git → deploys automatically.



Multi-Tenancy & Alerts

Tenant Isolation

```
kind: Kustomization
spec:
  serviceAccountName: team-alpha-sa
  sourceRef:
    kind: GitRepository
    name: team-alpha-repo
  path: ./deploy
  targetNamespace: team-alpha
  prune: true
```

- **serviceAccountName** — RBAC-scoped SA per team
- **targetNamespace** — force resources into one NS

Notification Alerts

```
kind: Provider
spec:
  type: slack
  channel: k8s-deployments
  secretRef: { name: slack-url }
---
kind: Alert
spec:
  providerRef: { name: slack-alerts }
  eventSeverity: error
  eventSources:
    - { kind: HelmRelease, name: "*" }
```

Monitor: Alert when `gotk_reconcile_condition{status="False"}` persists > 10 min. Grafana dashboard ID: **16714**.



Pipeline Tools

CI platforms, image scanning, and policy enforcement

GitHub Actions for Kubernetes

```
name: CI Pipeline
on: { push: { branches: [main] } }
jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/github-actions
          aws-region: us-east-2
      # ... Login to ECR, build, scan ...
      - name: Push and update GitOps
        run: |
          docker push $ECR_REGISTRY/myapp:${{ github.sha }}
      # ... kustomize edit set image, git commit & push
```

Jenkins X & Tekton

Tekton Pipelines

- Kubernetes-native CI/CD framework
- Pipelines run as Pods in the cluster
- CRDs: Task, Pipeline, PipelineRun
- Reusable task catalog (Tekton Hub)

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata: { name: build-image }
spec:
  steps:
    - name: build
      image: gcr.io/kaniko-project/executor
      args:
        - --destination=$(params.image)
        - --context=$(workspaces.source.path)
```

Jenkins X

- Opinionated CI/CD for Kubernetes
- Built on Tekton underneath
- Automated environment promotion
- Preview environments per PR
- GitOps-native from the start

Image Scanning in CI

Trivy

- Open-source (Aqua Security)
- OS packages + language deps
- IaC scanning + SBOM generation

```
# Fail pipeline on critical CVEs
trivy image --exit-code 1 \
  --severity CRITICAL \
  --ignore-unfixed myapp:v1.2.3

# Generate SBOM
trivy image --format spdx-json \
  -o sbom.json myapp:v1.2.3
```

Snyk

- Commercial with free tier
- Deep language dependency analysis
- Fix suggestions and PR automation

```
# Scan container image
snyk container test myapp:v1.2.3

# Monitor for new vulnerabilities
snyk container monitor \
  myapp:v1.2.3 --org=mycompany
```

Policy as Code

OPA Gatekeeper

- Admission controller (Rego language)
- ConstraintTemplates + Constraints

```
kind: K8sRequiredLabels
metadata: { name: require-team-label }
spec:
  match:
    kinds: [{ apiGroups: ["apps"], kinds: ["Deployment"] }
    parameters:
      labels: ["team", "cost-center"]
```

Kyverno

- Kubernetes-native (YAML, no Rego)
- Validate, mutate, generate resources

```
kind: ClusterPolicy
metadata: { name: disallow-latest-tag }
spec:
  validationFailureAction: Enforce
  rules:
    - name: require-image-tag
      match: { resources: { kinds: ["Pod"] } }
      validate:
        pattern:
          spec:
            containers:
              - image: "!*:latest"
```



Recommended Patterns

Strategies for reliable, auditable delivery

Environment Promotion Strategies

Promotion Flow

Dev → Staging → Pre-prod → Production

Branch-Based Promotion

- Each environment maps to a branch
- PR from dev → staging → main
- Simple to understand
- Merge conflicts can be challenging

Directory-Based Promotion

- Single branch, overlay directories
- PR updates overlay path (e.g., image tag)
- No branch divergence
- Promoted via automated PRs

Secrets Management

Sealed Secrets

- Encrypt with public key; decrypt only in-cluster
- Safe to commit to Git

```
kubeseal --format yaml \  
< secret.yaml \  
> sealed-secret.yaml
```

External Secrets Operator

- Sync from AWS SM / Vault
- Auto-rotation support

```
kind: ExternalSecret  
spec:  
  refreshInterval: 1h  
  secretStoreRef: { name: aws-secrets }  
  target: { name: db-creds }
```

SOPS

- Encrypts values, not keys -- KMS / PGP / age

```
sops --encrypt --kms arn:aws:kms:... secrets.yaml > secrets.enc.yaml
```

Rollback Strategies

Git Revert (Recommended)

- `git revert <commit>`
creates a new commit
- Preserves complete history
- ArgoCD/Flux auto-syncs the revert

```
# Revert a bad deployment
git revert abc123
git push origin main
# ArgoCD detects change and syncs
```

ArgoCD Manual Rollback

- Rollback via UI or CLI
- Temporary — next sync overwrites
- Use for emergencies only

```
# ArgoCD CLI rollback
argocd app history frontend
argocd app rollback frontend 3
```

GitOps rollback = Git revert. Forward-only commit history provides a clear, auditable record.

CI/CD Security Best Practices

Pipeline Security

- Use OIDC for cloud auth (no long-lived keys)
- Pin action versions by SHA, not tag
- Restrict self-hosted runner permissions
- Require signed commits in GitOps repos
- Separate CI and CD credentials

Supply Chain Security

- Sign images with Cosign / Sigstore
- Generate and verify SBOMs
- Use admission policies to verify signatures
- Implement SLSA provenance
- Audit base image dependencies

Defense in depth: Scan in CI, enforce policies at admission, monitor at

Putting It All Together

Complete GitOps Pipeline

1. Developer pushes code to app repo
2. CI pipeline: build → test → scan → sign → push image
3. CI updates image tag in GitOps repo (automated PR)
4. PR reviewed and merged
5. ArgoCD/Flux detects change, syncs to cluster
6. Health checks verify deployment
7. Notification sent (Slack, PagerDuty)

Benefits achieved:

Declarative, auditable,
reproducible, self-healing, secure

Key repos: Application repo
(source code) + GitOps repo
(manifests) = clean separation
of concerns



Module Summary

- GitOps: Git as single source of truth for declarative infrastructure
- ArgoCD and FluxCD reconcile cluster state with Git
- Directory-based promotion with Kustomize overlays avoids branch divergence
- Secrets: Sealed Secrets, ESO, or SOPS
- Rollback via git revert preserves audit history
- Supply chain security: scanning, signing, admission policies



Lab 13: GitOps with ArgoCD

Estimated Time: ~30 minutes

- Explore the ArgoCD UI and CLI
- Create an ArgoCD Application from a public Git repo
- Observe sync status and application health
- Make a change, trigger a sync, and perform a rollback
- *Optional:* Explore FluxCD reconciliation and drift detection

Key Takeaways

Remember These Points

1. Separate app repos from GitOps manifest repos
2. Enable drift detection and self-healing so clusters always match Git
3. Use OIDC for CI auth and pin action versions by SHA

? Questions & Discussion

Module 13: CI/CD Integration & GitOps

Final module — Course Wrap-Up